

第三章 软件需求分析 AI讲解

开篇导读

一句话概括：需求分析的任务是搞清楚"系统到底要做什么"，并建立系统的逻辑模型——它把用户模糊的"想要"转化为开发能用的"规格"。

本章在考试中的地位：中等频率考点章。高频考点是需求分类（功能/非功能/领域）的场景判断、三种模型分别用什么图、用例图的包含(include)与扩展(extend)关系、需求验证的四个维度。本章的DFD和数据字典与第二章衔接，结构化分析方法是传统考点。

一、需求分析的任务

核心概念

需求分析的根本任务是确定系统必须完成什么工作，也就是确定系统必须具备的功能和性能。它的输出是一个完整的逻辑模型——描述"系统做什么"，而不是"怎么做"。

为什么重要

需求分析是软件开发的"地基"。统计表明，软件项目中约40%-60%的错误源于需求阶段，而需求错误到后期修复的成本是需求阶段的50-200倍。需求分析的口号是："做正确的事"比"正确地做事"更重要。

易混辨析

考法	易错点
"需求分析确定什么"	确定"做什么"（逻辑模型），不是"怎么做"（物理模型/设计方案）
"需求分析 vs 可行性研究"	可行性研究是简化压缩版，判断"做不做"；需求分析是详细版，确定"具体做什么"

二、需求获取方法

核心概念

需求获取是需求分析的第一步，要从用户那里收集需求。常用方法：

方法	特点	适用场景
访谈	结构化/非结构化对话	深入了解个别用户需求
问卷调查	大范围收集	用户群体大、需求分散
情景分析	用用例/用户故事描述场景	理解业务流程
快速原型	建原型让用户体验	需求模糊、用户说不清
观察法	现场跟踪记录	用户自己都说不清流程时



五种方法深度拆解

1. 访谈 (Interview)

- 是什么：与用户/利益相关者面对面（或视频）对话，提问获取需求
- 机制/类型：
 - 结构化访谈：事先列好问题清单，按顺序问（适合需求边界清晰）
 - 非结构化访谈：开放式聊天，跟着用户思路走（适合早期探索）
 - 半结构化访谈：核心问题预定+随机追问（最常用，平衡深度和广度）
- 驱动方式：对话驱动（通过双向交流挖掘隐性需求）
- 优点：能深入挖掘背景动机；可即时澄清歧义；建立人际信任
- 缺点：耗时（每人1-2小时）；用户言行不一（说的是A，实际做的是B）；访谈员主观性影响结果
- 适用场景：关键用户、决策者、领域专家；项目早期需求探索阶段
- 技巧：先开放问题（“你平时怎么处理订单？”）后封闭问题（“是否需要批量导出功能？”）；让用户演示而非只描述

恍然大悟：访谈最大的陷阱是“用户说他想要快马，其实他想要的是更快到达”（亨利·福特名言）。

访谈员要追问“为什么需要这个功能”而不是只记录“用户要什么”——表面需求往往不是真实需求。

2. 问卷调查 (Questionnaire)

- 是什么：设计标准化问题集，发给大量用户填写，统计分析结果
- 机制：
 1. 明确调研目标（如“统计各功能使用频率”）
 2. 设计问题（封闭题为主：单选、多选、量表评分）
 3. 发放回收（线上工具或纸质）
 4. 数据统计分析（频次、占比、相关性）
 - 优点：覆盖面广（可触达上千用户）；结果量化便于统计；成本低（边际成本接近零）
 - 缺点：

1. 问题设计难——问题歧义会导致数据失真

2. 回收率低——通常10-30%，不填的用户可能恰好是关键用户

3. 无法追问——用户答“很重要”但不知具体多重重要

4. 难发现意外需求——只能测预设问题，问不出“未知的未知”

- 适用场景：用户量大（>100人）、需求分散；需要量化数据支撑决策
- 典型例子：B端产品调研——给所有部门发问卷统计“哪些功能用得最多/最少”

恍然大悟：问卷与访谈是广度vs深度的取舍——问卷能覆盖1000人但每人只能问20个问题，访谈只能覆盖10人但每人能问100个深问题。所以最佳实践是先访谈定方向，后问卷做验证：访谈出假设→问卷大规模验证。

3. 情景分析 (Scenario Analysis)

- 是什么：通过具体的使用场景描述，让用户/开发者站在用户角度演练业务流程
- 机制/形式：
 - 用例 (Use Case)：详细描述“用户A为完成目标B，与系统进行的交互序列”（含主流程、扩展流程、异常流程）
 - 用户故事 (User Story)：敏捷开发中的简短格式：“作为<角色>，我想要<功能>，以便<价值>”
 - 故事板 (Storyboard)：用画面/线框图分镜表达使用场景
- 优点：以用户视角而非系统视角思考；自然带出非功能需求（性能、易用性）；便于和非技术人员



沟通

- 缺点：场景写不全易漏需求；过于详细易陷入实现细节；需大量场景才能覆盖完整功能
 - 适用场景：需求分析中期；面向用户的应用系统（B端C端均适用）
 - 典型例子：电商"加入购物车"用例——前置条件（已登录）→主流程（点击按钮→加入成功）→扩展（库存不足提示）→异常（网络超时）
- 恍然大悟：情景分析的本质是"逼着你想清楚边界"——只写"用户能加入购物车"是空话，写出"未登录如何处理？库存为0如何处理？已加过如何处理？"才是真需求。场景越具体，遗漏越少。这也是为什么用例必须包含扩展流程和异常流程，而非只写主流程。

4. 快速原型（Prototyping）

- 是什么：快速搭建可交互的原型（界面草图、可点击Demo），让用户体验后反馈需求
- 与第一章过程模型的关联：这里的"快速原型"是需求获取技术，第一章的"快速原型模型"是整个开发模型。前者只生成原型用于沟通，后者是用原型驱动整个开发过程
- 原型类型：
 - 抛弃型原型：只用于澄清需求，最终丢弃重写（最常用）
 - 演化型原型：原型逐步演化成最终产品（敏捷开发常见）
- 保真度分级：低保真（线框图/纸质）→中保真（Axure/Figma）→高保真（可点击Demo）
- 优点：直观（用户看得见摸得着）；早期暴露需求误解；减少返工
- 缺点：用户易把原型当成品（造成期望偏差）；做原型本身耗时；可能限制设计创新
- 适用场景：需求模糊；用户描述能力差；新业务领域；UI/UX密集的产品
- 典型例子：给餐饮老板做点餐系统——画3张界面草图（菜单页/下单页/账单页）→老板边看边说"这里加个备注栏"→改图再确认

恍然大悟：原型的核心价值不是"做出像样的Demo"，而是"把用户头脑里的隐性需求逼出来"。用户看到原型才会说"哦不对，应该是这样"——人对具体事物的批判力远强于对抽象描述的想象力。所以原型越早越糙越好，越早暴露分歧越省钱。

5. 观察法（Observation）

- 是什么：到用户实际工作现场观察记录，捕捉"说不清写不出"的隐性流程
- 机制/类型：
 - 被动观察：在旁边看不打扰（捕捉真实操作）
 - 参与式观察：观察者参与工作（深度理解但耗时长）
 - 影子跟随：跟着一个用户一整天（捕捉完整工作流）
- 优点：
 1. 发现用户说不清的隐性流程（如老员工的快捷操作）
 2. 验证用户描述的真实性（说的vs做的差异）
 3. 发现工作环境的影响（噪音、中断、并行任务）
- 缺点：
 1. 霍桑效应——被观察者会改变行为（"演给你看"）
 2. 耗时长——通常需观察1-3天才能看到典型流程
 3. 难以观察非常规情况——突发事件可能没遇上
 - 适用场景：用户操作流程复杂且依赖经验（如医生、工厂工人、客服）；现有系统替换升级
 - 典型例子：替换医院HIS系统——观察医生看诊3天，发现真实流程是"边问诊边瞄电脑边写处方"，三件事并行——这点用户访谈绝对说不出来

恍然大悟：观察法是5种方法中唯一"看做了什么"而非"听说了什么"的方法。其他4种都依赖用户的语言表达能力，但人对自己习以为常的操作往往说不清——就像让你描述"如何骑自行车"，没人能说全所



有平衡微调动作。观察法补的就是这个盲区。

五种方法的整体定位

方法	信息获取方式	关键词
访谈	问	深度对话
问卷	写	广度统计
情景分析	演	场景演练
快速原型	试	体验反馈
观察法	看	现场捕捉

总恍然大悟：5种方法对应"问、写、演、试、看"五种获取方式，互相补充而非替代。实际项目通常组合使用：访谈关键用户定方向 → 问卷大规模验证 → 情景分析厘清流程 → 快速原型确认理解 → 观察法补隐性需求。没有一种方法能单独获取完整需求——每种方法都有盲区，组合使用才能拼出全貌。

记忆技巧

口诀："访问情原观"（访谈、问卷、情景、原型、观察）——谐音"访问情缘观"。
五字诀："问写演试看"——5种方法的核心动作。

三、需求分类（高频考点）

核心概念

需求分为三大类：

需求类型	含义	例子
功能需求	系统必须做什么（功能/行为）	"系统应支持用户登录""系统应能计算订单总价"
非功能需求	系统做得怎么样（性能/质量属性）	"响应时间<2秒""可用性99.9%""支持1000并发"
领域需求	来自应用领域的特定需求	银行系统的"日终结算"规则、医疗系统的"电子病历标准"

为什么重要

这是选择题最爱考的点——给出一个需求描述，判断它属于哪类。设坑点是功能需求 vs 非功能需求的区分，以及设计约束的归类。

易混辨析（重点中的重点）

需求描述	属于哪类	区分关键
"系统支持用户注册登录"	功能需求	描述"做什么"（动作/功能）
"系统响应时间不超过2秒"	非功能需求	描述"做得怎么样"（性能/质量）
"系统必须用Java开发"	设计约束（属非功能需求范畴）	描述"用什么做"（技术限制）
"系统需符合金融监管要求"	领域需求	来自特定行业/领域

功能 vs 非功能的区分口诀：

- 功能需求回答"做什么"（动词开头：登录、计算、查询）

- 非功能需求回答"做得多好"（性能、可靠性、安全性、可维护性等质量属性）

非功能需求详细子分类（必背）

非功能需求是"质量属性"的总称，常见子类（FURPS+模型/ISO 25010）：

子类	含义	量化指标举例
性能（Performance）	系统响应速度、吞吐量	响应时间 ≤ 2 秒；吞吐量 ≥ 1000 TPS；并发用户 ≥ 10000
可靠性（Reliability）	系统正常运行能力	可用性99.9%（年宕机 < 8.76 h）；MTBF（平均故障间隔） > 1000 h
安全性（Security）	数据保密、完整、可用	CIA三要素：保密性Confidentiality、完整性Integrity、可用性Availability
易用性（Usability）	用户使用难易度	新用户首次使用5分钟内完成下单；满意度评分 > 4.5
可维护性（Maintainability）	修改/扩展的便利度	平均修复时间MTTR < 2 小时；模块独立性高
可移植性（Portability）	跨平台/环境能力	支持Windows/Linux/macOS；支持MySQL/Oracle
可扩展性（Scalability）	处理增长负载的能力	用户从1万扩到100万性能不显著下降

非功能需求的"量化"原则：

- 反例："系统应运行快" → 不可验证
- 正例："首页加载时间在4G网络下 ≤ 2 秒" → 可测量

领域需求详细子分类

子类	含义	例子
业务领域规则	来自具体业务的固有规则	银行T+1结算；股票涨跌停板10%；保险条款约定
行业标准	来自行业规范/法规	医疗HL7数据交换标准；金融PCI-DSS安全标准；电信3GPP协议
法律法规	来自国家/地区法律	数据保护法（GDPR/个保法）；电子签名法；会计准则

领域需求的特点：

- 不可协商（违反就违法/违规）
- 跨产品稳定（同行业不同产品都要遵守）
- 项目早期就要识别（晚发现成本极高）

三类需求的优先级处理

实际项目中三类需求的处理顺序：

1. 领域需求最先识别——是底线，违反就GG
2. 功能需求次之——是产品价值的来源
3. 非功能需求贯穿全程——影响架构设计，但具体指标可在设计阶段细化

恍然大悟：功能需求是软件的"技能"，非功能需求是软件的"体质"，领域需求是软件的"户籍"——技能差可以训练（重写代码），体质差很难补救（架构改动伤筋动骨），户籍是天生的（违反就出局）。新手项目常犯的错是只盯功能需求（"系统能登录、能下单"），忽略非功能（"登录要多快？"），结果上线后用户骂"卡得要死"——功能100%实现但用户不买账。这也是为什么需求评审必须强制问"非



功能指标是多少”。

记忆技巧

口诀：“功能做什么，非功做得好，领域看行业”——功能需求是“做什么”，非功能需求是“做得多好”，领域需求看“哪个行业”。

四、三种模型（必背对应关系）

核心概念

需求分析要建立系统的三种模型，每种模型用不同的图来描述：

模型	描述什么	用什么图
数据模型	系统有哪些数据、数据间关系	实体-联系图（ER图）
功能模型	系统做什么、数据怎么流动	数据流图（DFD）
行为模型	系统的状态及状态转换	状态转换图（STD）

为什么重要

这是高频考点，出题方式是问某种模型用什么图，或者问某种图描述什么模型。三者对应关系必须牢记。

易混辨析

考法	易错点
"功能模型用什么图"	用DFD（数据流图），不是ER图，不是状态转换图
"数据模型用什么图"	用ER图（实体-联系图），不是DFD
"行为模型用什么图"	用状态转换图（STD），不是DFD

记忆技巧

对应口诀：“数功D行状态”——数据模型用ER图，功能模型用DFD，行为模型用状态转换图。

ER图详解（数据模型，画图题高频考点）

ER图（Entity-Relationship Diagram）用三种图形描述数据模型：

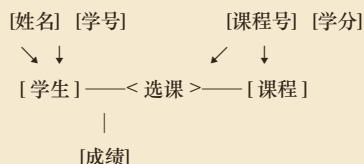
图形	表示什么	语义角色
矩形	实体	"名词"（如学生、课程）
椭圆	属性	"修饰"（如姓名、年龄）
菱形	联系	"动词"（如选课、选修）

三种联系类型：

类型	含义	例子
1:1	一对一	一个丈夫对应一个妻子
1:N	一对多	一个班级对应多个学生
M:N	多对多	多个学生选多门课程

完整例子（学生选课系统）：





- 学生（矩形）有姓名、学号属性（椭圆）
- 课程（矩形）有课程号、学分属性（椭圆）
- 选课（菱形）是学生和课程的联系，联系本身也有属性"成绩"（椭圆）
- 学生和课程是M:N关系（一个学生选多门课，一门课被多个学生选）

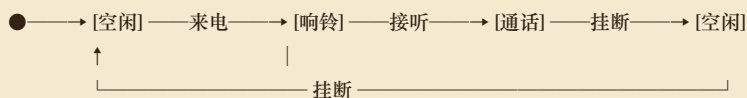
恍然大悟：为什么用三种图形？因为实体是"名词"（矩形像表格，代表数据表）、属性是"修饰"（椭圆像气泡挂在实体上）、联系是"动词"（菱形像连接器把两个实体连起来）——图形本身就暗示了语义，看图就能读懂"谁和谁有什么关系"。

状态转换图详解（行为模型，画图题考点）

状态转换图（STD，State Transition Diagram）描述系统的动态行为：

要素	含义	符号
状态	系统在某一时刻的状况	圆角矩形
事件	触发状态转换的动作	箭头上的标签
转换	从一个状态变到另一个状态	箭头
初态	系统的起始状态	实心圆
终态	系统的结束状态	同心圆（带边框的圆）

完整例子（电话被叫状态）：



- 初态：实心圆 ●
- 空闲→响铃→通话 是中间状态（圆角矩形）
- 事件：来电、接听、挂断（箭头标签）
- 无终态（电话一直循环使用）

恍然大悟：状态转换图为什么重要？因为它描述"系统在什么状态下收到什么事件会变成什么状态"——这是动态行为的精确描述。DFD只画静态数据流（数据从哪到哪），画不出"状态变化"（如电话从空闲变响铃）。ER图只画静态数据结构（有哪些实体），也画不出"状态转换"。三种图各管一摊：ER图管数据（静态结构），DFD管功能（数据流动），STD管行为（状态变化）。

谐音记忆："数（树）E，功（工）D，行（行）状态"——树用E画，工厂用D画，行人看状态。

五、结构化分析（SA）

核心概念

结构化分析（SA，Structured Analysis）是一种传统的、面向数据流的需求分析方法。它的核心工具是：

- 数据流图（DFD）——描述功能模型
- 数据字典（DD）——定义数据



- 实体-联系图（ER图）——描述数据模型
- 状态转换图（STD）——描述行为模型

SA方法论深度拆解

- 是什么：以"数据流动"为核心抽象的需求分析方法，自顶向下逐层分解系统功能，配套四种图形工具描述系统的功能、数据、行为
- 方法论步骤（必背）：
 1. 理解现状：调研现有业务流程（用系统流程图描述物理系统）
 2. 画顶层DFD：把整个系统看作一个处理，标出外部实体和主要数据流
 3. 分层细化DFD：自顶向下逐层分解（0层→1层→2层），每层关注当前抽象层次
 4. 建数据字典：定义DFD中所有数据流、数据项、数据存储、处理的精确含义
 5. 建ER图：从数据存储中识别实体、联系、属性，描述数据模型
 6. 建状态转换图：对有状态的对象（如订单、用户账户）描述状态变迁
 7. 整合形成SRS：把上述四张图+文字说明组织成需求规格说明书
 - 驱动方式：数据流驱动（以数据流动为线索贯穿全程）
 - 优点：
 1. 结构清晰——分层分解适合大系统
 2. 文档规范——四种图形工具配合文字说明，标准化程度高
 3. 易培训易复用——方法论成熟，新人按步骤就能做
 - 缺点：
 1. 不适合需求多变——文档驱动，需求变更要重画所有图
 2. 与OO衔接差——SA输出的是过程化模型，转OO设计要重新建模
 3. 抽象层次单一——只擅长描述数据处理，不擅长描述对象行为和交互
 - 适用场景：传统信息系统（数据处理为主的系统，如银行、保险、ERP）；瀑布或RUP开发模型

易混辨析

考法	易错点
"SA方法的工具"	DFD + DD + ER图 + 状态转换图（四个工具，不要漏）
"SA是面向什么的方法"	面向数据流的方法（不是面向对象、不是面向数据结构）
"SA与OO的本质区别"	SA以"数据流动"为核心，OO以"对象"为核心

恍然大悟：SA方法的精髓是把"数据流动"当作系统的灵魂——所有工具都围绕数据展开。DFD画数据怎么流，DD定义数据是什么，ER图刻画数据间的关系，STD描述数据驱动的状态变化。这是结构化思维的本质：用'数据'统一一切，把世界看作数据加工厂。OO方法的精髓则是把"对象"当作灵魂，用对象的属性+方法封装一切。两者世界观不同，注定方法不同。SA适合"数据加工型"业务（订单流转、报表生成），OO适合"实体交互型"业务（角色协作、行为复杂）。SA虽然在新项目里被OO逐渐替代，但其"自顶向下分而治之"的思想已经融入所有现代方法。

六、需求规格说明书（SRS）

核心概念



需求规格说明书（SRS，Software Requirements Specification）是需求分析阶段的输出文档，它详细描述了系统的功能需求、非功能需求和约束。

SRS的特性

- 一份好的SRS应该具备：
- 正确性：准确反映用户需求
 - 无歧义性：每条需求只有一种解释
 - 完整性：涵盖所有需求
 - 可验证性：每条需求都能被验证
 - 一致性：需求之间不矛盾
 - 可修改性：便于修改
 - 可跟踪性：能追溯到来源

七大特性深度拆解（每条配反例+检查方法）

特性	反例（坏需求）	正例（好需求）	检查方法
正确性	"系统应满足用户所有需求" (空话)	"系统应支持用户用手机号+ 密码登录"	与用户/原始需求文档逐条对照
无歧义性	"系统应快速响应" (“快速”模糊)	"用户点击查询按钮后，结果 应在2秒内返回"	用具体数字/术语替换"快/好/ 友好"等形容词
完整性	只写了正常流程，未写异常 处理（如登录失败、网络中 断）	主流程+扩展流程+异常流程 齐全	用需求覆盖矩阵检查每个用 例的所有路径
可验证性	"系统应该用户友好" (怎么验证?)	"新用户首次使用，5分钟内 能完成下单"	问"如何测?"——能写出测 试用例就可验证
一致性	需求A: "订单24小时内可取 消"; 需求B: "下单后立即扣 款不退" (前后矛盾)	A和B约束相同的处理逻辑	交叉对比检查 (人工评审 + 工具辅助)
可修改性	同一需求散落在多处，改一 处会漏 ("用户可批量导出" 在需求12和需求48都写了)	每条需求只在一处定义，其 他地方引用	编号、引用、避免重复
可跟踪性	需求来源不明 ("系统要支持 微信支付"——谁提的?)	每条需求标注来源 (如"需求 #23来源: 客户访谈2023-05-1 0王经理")	需求追踪矩阵 (向后追实现 ，向前追来源)

七特性的内在关系

- 正确性 + 完整性：约束需求"够不够"（覆盖度）
- 无歧义性 + 一致性：约束需求"清不清"（清晰度）
- 可验证性：约束需求"能不能测"（可测性）
- 可修改性 + 可跟踪性：约束需求"好不好维护"（管理性）

恍然大悟：七特性的核心目的不是写出"漂亮的文档"，而是让需求成为可验收的合同——每条需求都要能被打勾（已实现）或打叉（未实现）。"系统应该用户友好"这种话写在合同里就是要流氓——开发说"已实现"，用户说"不友好"，谁说了算？所以无歧义性和可验证性是七特性的核心，其他都是辅助。一份合格的SRS，每条需求都该能直接转成测试用例。

易混辨析



考法	易错点
"SRS是什么阶段的输出"	需求分析阶段（不是可行性研究、不是设计阶段）
"SRS描述什么"	描述"做什么"（逻辑模型），不描述"怎么做"

七、需求验证（四个维度）

核心概念

需求分析完成后，需要验证需求的质量。验证从四个维度：

维度	含义	通俗解释
一致性	需求之间不矛盾	"不能前后打架"
现实性	现有技术能实现	"不能异想天开"
完整性	涵盖所有需要的需求	"不能漏掉重要的"
有效性	确实解决用户问题	"不能答非所问"

四维度的验证手段（必背）

每个维度对应不同的失败模式和验证方法：

维度	对应失败模式	具体验证手段	输出
一致性	自相矛盾	交叉对比检查：把需求两两对比，看是否冲突；用工具检查同名实体的描述是否一致	矛盾需求清单
现实性	做不出来	技术可行性评审：技术专家评估各需求的实现难度；可做技术原型验证	高风险需求清单
完整性	漏需求	需求覆盖矩阵检查：检查每个用户角色、每个业务场景是否都有对应需求；和原始访谈记录对照	需求覆盖率（缺口列表）
有效性	答非所问	用户走查/原型确认：让用户审阅需求或体验原型，确认"这就是我想要的"	用户签字确认

四维度与SRS七特性的关系

需求验证的"四维度"是对SRS七特性的检查：

验证维度	对应SRS特性
一致性验证	SRS的"一致性"特性
现实性验证	（隐含的可实现性，七特性外的补充）
完整性验证	SRS的"完整性"特性
有效性验证	SRS的"正确性"特性

恍然大悟：四维度对应四个不同的失败模式——自相矛盾（一致性问题）、做不出来（现实性问题）、漏需求（完整性问题）、答非所问（有效性问题）。每个失败模式有不同根因，所以验证手段也不同：矛盾要靠对比检查，难做要靠技术评审，漏需求要靠覆盖矩阵，答非所问要靠用户确认。如果只用一种手段（如只做用户评审）一定会漏问题——评审能发现"答非所问"，但发现不了"自相矛盾"或"技术做不出"。这也是为什么需求验证必须四维度都做，缺一不可。

易混辨析

考法	易错点
"需求验证的维度"	一致性、现实性、完整性、有效性（四个！不要和SRS特性混淆）
"一致性 vs 完整性"	一致性是"不矛盾"，完整性是"不遗漏"

记忆技巧

口诀："一现完有"（一致性、现实性、完整性、有效性）——谐音"一线全有"：一线（一致性、现实性）全有（完整性、有效性）。

八、UML基础（用例图）

核心概念

UML（统一建模语言）是面向对象方法的标准建模语言。需求分析阶段最常用的是用例图。

用例图的组成

- 参与者（Actor）：与系统交互的人或外部系统（用小人表示）
- 用例（Use Case）：系统提供的功能（用椭圆表示）
- 关系：
 - 关联：参与者与用例之间的连线
 - 包含（include）：一个用例必然包含另一个用例的行为
 - 扩展（extend）：一个用例在特定条件下扩展另一个用例的行为
 - 泛化：用例之间的继承关系

包含(include) vs 扩展(extend)（必考辨析）

关系	含义	特点	例子
包含(include)	A必然执行B	基用例A依赖B，B一定执行	"下单"包含"登录"（下单必须先登录）
扩展(extend)	A在特定条件下执行B	扩展用例B可选执行	"下单"扩展"使用优惠券"（下单时可选使用优惠券）

区分关键：

- 包含是"必须有"——基用例离不开被包含用例
- 扩展是"可以有"——扩展用例是可选的附加行为

易混辨析

考法	易错点
"登录被所有功能用例包含"	包含关系（登录是必须的）



"支付时可选使用积分抵扣"	扩展关系（积分抵扣是可选的）
"包含是基用例依赖被包含用例"	正确！基用例离不开被包含用例

记忆技巧

口诀："包含必有，扩展可有"——包含（include）是"必然执行"，扩展（extend）是"条件执行/可选"。

类比：包含像"吃饭必须用筷子"（必须有），扩展像"吃饭可以喝饮料"（可选）。

面向对象基本概念（UML的基础）

UML是面向对象方法的建模语言，理解UML前先掌握面向对象的核心概念：

概念	含义	通俗解释
类	具有相同属性和行为的对象的抽象模板	"图纸"（如"学生"类）
对象	类的实例	"按图纸造出的实物"（如"张三"这个学生）
封装	隐藏内部细节，只暴露接口	"电视只露按钮，内部电路藏起来"
继承	子类继承父类的属性和方法	"儿子继承父亲的财产"
多态	同一消息不同对象有不同响应	"叫'叫一声'，狗汪汪、猫喵喵"
消息	对象间的通信	"对象A调用对象B的方法"

六大概念深度拆解

1. 类（Class）

- 是什么：具有相同属性和行为的对象的抽象模板（蓝图）
- 组成：类名 + 属性（数据成员）+ 方法（行为成员）
- 例子：class Student { 属性：姓名、学号、年龄；方法：选课()、交学费() }
- 特点：类本身不占内存，只是定义；只有实例化为对象才占内存

2. 对象（Object）

- 是什么：类的具体实例，是程序运行时的实体
- 特征：每个对象有自己独立的属性值，但共享类的方法定义
- 例子：Student zhangsan = new Student("张三", "20231001", 19), Student lisi = new Student("李四", "20231002", 20)——两个对象用同一个类，但属性值不同
- 类与对象的关系：类是"图纸"，对象是"按图纸造出的实物"

3. 封装（Encapsulation）

- 是什么：把数据（属性）和操作数据的方法捆绑在类内，对外隐藏内部实现细节，只暴露公开接口
- 机制：通过访问修饰符控制可见性
- private（私有）：只有类内部能访问（默认推荐）
- protected（保护）：类内部和子类能访问
- public（公开）：任何地方都能访问
- 反例：class Account { public double balance; } → account.balance = -1000 任何人都能直接改余额
- 正例：class Account { private double balance; public void deposit(double amount) { if(amount>0) balance+=amount; } } → 只能通过deposit方法存钱，且有合法性校验
- 优点：

1. 修改自由——内部实现可以随时改，只要接口不变，外部代码不受影响
2. 数据安全——防止数据被非法修改



3. 降低耦合——使用方只需知道接口，不需了解内部

- 核心原则：信息隐藏（Information Hiding）——把可能变化的设计决策隐藏在类内

4. 继承（Inheritance）

- 是什么：子类（派生类）自动获得父类（基类）的属性和方法，并可扩展或覆盖
- 机制：
- 单继承：子类只能有一个父类（Java、C#）
- 多继承：子类可有多个父类（C++）；多继承会引发"菱形继承"问题
- 接口实现：单继承语言中通过实现多个接口实现"伪多继承"
- 关系语义：IS-A（是一种）——Dog IS-A Animal
- 例子：

```
class Animal { void eat(){...} }    // 父类
class Dog extends Animal { void bark(){...} } // 子类继承eat(), 新增bark()
```

- 优点：复用代码（子类不用重写父类方法）；建立类层次结构（便于理解和维护）
- 缺点：紧耦合（父类改动影响所有子类）；继承层次过深难以维护
- 使用建议："优先组合而非继承"（Composition over Inheritance）——只在真正IS-A关系时用继承

5. 多态（Polymorphism）

- 是什么：同一消息（方法调用）在不同对象上产生不同的行为
- 实现机制：通过"继承+方法重写（Override）"实现
- 典型例子：

```
Animal a1 = new Dog(); a1.speak(); // 输出"汪汪"
Animal a2 = new Cat(); a2.speak(); // 输出"喵喵"
// 调用代码完全相同，但行为不同——这就是多态
```

- 优点：

1. 支持开闭原则——增加新动物类型不需修改原有代码

2. 解耦使用方与具体类——使用方只依赖父类Animal，不关心具体是Dog还是Cat

3. 代码灵活可扩展——同一接口可处理多种类型

- 三种多态形式：
- 继承多态（最常见，通过方法重写）
- 接口多态（实现同一接口的不同类）
- 重载多态（同名方法不同参数，编译期决定）

6. 消息（Message）

- 是什么：对象之间通过调用方法进行的通信
- 形式：接收者对象.方法名(参数) 或 UML顺序图中的箭头
- 三要素：

1. 接收者——调用谁的方法

2. 方法名——做什么操作

3. 参数——附带的信息

- 例子：student.selectCourse("数据库") → 向student对象发送"选课"消息，参数是课程名
- 意义：消息是OO系统中对象协作的唯一方式——对象不直接读写其他对象的数据，只通过消息交互

三大支柱的因果关系

封装、继承、多态是OO的三大支柱：



恍然大悟：封装隔离变化、继承复用代码、多态扩展行为——三者支撑了OO的核心价值"扩展性"。封装是基础（没有封装，类内部全裸暴露，继承和多态都没意义）；继承是手段（让类形成层次，子类复用父类代码）；多态是目的（让代码能处理"未来才会出现的新类型"，不改老代码加新功能）。这就是"开闭原则"（对扩展开放、对修改关闭）的实现路径。面向过程做不到这点——加新功能必须改原函数，原函数被改就可能破坏老功能；OO通过多态让"加新类"代替"改老代码"，这是OO相比面向过程的本质优势。

易混辨析：面向对象为什么比面向过程更适合大项目？因为面向对象用"类"把数据和操作数据的方法绑在一起（封装），改一个类不影响其他类；面向过程的数据和函数是分开的，改一个数据结构所有用到它的函数都要改。这就是"高内聚低耦合"在编程范式层面的体现。

UML主要图分类（结构图 vs 行为图）

UML图分两大类，共9种（UML 1.x）或13种（UML 2.0）：

类别	图名	描述什么	考试频率
结构图（静态）	类图	类的属性、方法及类间关系	★★高频
	对象图	某时刻对象的快照	低
	组件图	系统组件及依赖	低
	部署图	硬件节点及部署	低
行为图（动态）	用例图	系统功能与参与者交互	★★★★高频
	顺序图	对象间消息的时间顺序	★★高频
	状态图	对象状态及转换(=状态转换图)	★★
	活动图	工作流/业务流程	★
	协作图	对象间消息（强调结构）	低

九种图的角色定位（含冷门图场景）

图名	关注点	典型应用场景
类图（★★）	类的结构、关系	详细设计——刻画系统的静态骨架
对象图（低）	某时刻对象的实例和值	调试时画对象快照；解释类图实例化后的样子
组件图（低）	软件组件（DLL、JAR、模块）及依赖	架构设计——展示系统由哪些可部署单元组成
部署图（低）	硬件节点及部署关系	运维设计——说明软件部署在哪些服务器、网络拓扑
用例图（★★★★）	用户视角的系统功能	需求分析——客户最容易看懂的图
顺序图（★★）	对象交互的时间顺序	详细设计——刻画一次业务流程中对象间的消息时序
状态图（★★）	单个对象的状态变化	需求/设计——刻画订单、账户等有状态对象的生命周期
活动图（★）	业务流程/工作流	业务建模——类似流程图但支持并发、分支、合并
协作图（低）	对象交互（按对象关系组织）	与顺序图等价但侧重"谁和谁说话"而非"先后顺序"，UML 2.0改名"通信图"

三组易混图的区分



易混对	区别
顺序图 vs 协作图	同样描述对象交互，顺序图按时间组织（垂直时间轴），协作图按对象关系组织（空间网络）
状态图 vs 活动图	状态图描述单个对象的状态变迁；活动图描述整个业务流程（可跨多个对象）
组件图 vs 部署图	组件图描述软件单元（什么模块）；部署图描述硬件部署（在哪台机器）

恍然大悟：UML九种图本质是从9个不同角度看同一个系统——结构图看"有什么"（静态骨架），行为图看"怎么动"（动态行为）；用例图看"用户视角"，顺序图看"对象视角"，部署图看"运维视角"。没有一张图能描述完整系统——就像盲人摸象，每张图只摸一部分。所以实际项目要画多张图组合，而非只画一张。考试只考3-4张高频图（用例、类、顺序、状态），其他图了解即可。

类图详解（高频考点）

类图描述系统中的类、类的属性和方法、以及类之间的关系。

类的表示：用矩形分三层——类名（顶层）、属性（中层）、方法（底层）

Student	← 类名
- name: String	← 属性（-表示私有）
- age: int	
+ getName(): String	← 方法（+表示公有）
+ study(): void	

类间的五种关系（从弱到强，必背）：

关系	含义	例子	记忆
依赖	A用到了B，但B变化会影响A	"学生"依赖"教材"	"用一下"
关联	A拥有B的引用（长期关系）	"学生"关联"课程"	"拥有"
聚合	整体与部分（部分可独立存在）	"班级"聚合"学生"	"可分离的包含"
组合	整体与部分（部分不能独立存在）	"人"组合"心脏"	"不可分离的包含"
继承(泛化)	子类继承父类	"大学生"继承"学生"	"is-a"

注：UML 中还有第6种关系"实现"（接口实现）——类实现接口，可视为继承的特殊形式（继承一个抽象规范），符号详见下表。

UML符号图形对应（必背）

关系	线型	箭头/形状	完整符号
依赖	虚线	开放箭头	A - - - - - > B
关联	实线	开放箭头（或无箭头）	A ———— > B
聚合	实线	空心菱形（整体端）	A ◇ ———— B（A是整体）
组合	实线	实心菱形（整体端）	A ◆ ———— B（A是整体）
继承（泛化）	实线	空心三角箭头（父类端）	A ———— △ B（B是父类）



实现（接口）	虚线	空心三角箭头（接口端）	A -----△ I（I是接口）
--------	----	-------------	------------------

记忆口诀：

- 菱形=整体：菱形画在"包含方"端，空心可分（聚合），实心不可分（组合）
- 三角=父类：三角画在"被继承方"端，实线是继承类、虚线是实现接口
- 虚实之分：虚线表示"弱关系/编译期关系"（依赖、实现），实线表示"强关系/运行期持有"（关联、聚合、组合、继承）

多重性（Multiplicity）

关联线两端可标注多重性，表示关联两端各有多少对象参与：

多重性	含义	例子
1	必有1个	一个订单必有1个客户
0..1	0或1个	一个员工可有0或1个工卡
* 或 0..*	0到多个	一个客户有0到多个订单
1..*	至少1个	一个班级至少1个学生
m..n	m到n个	一个团队2到5人：2..5

例子：Customer 1 ——— 0..* Order —— 一个客户对应0到多个订单，一个订单对应1个客户。

五种关系强度递增

依赖 < 关联 < 聚合 < 组合 < 继承

关系	生命周期耦合	持有方式	强度
依赖	临时（方法调用时）	不持有	最弱
关联	长期	持有引用，但独立创建	弱
聚合	长期	持有引用，整体不负责创建/销毁部分	中
组合	同生共死	持有引用，整体负责创建和销毁部分	强
继承	类型层面	子类是父类的特例	最强

注：继承是类型层面关系，与前4种实例层面关系不完全可比，此处仅按耦合/牵连度排序。"实现关系"可视为继承的接口版本。

恍然大悟：聚合vs组合的区别是什么？聚合是"可分离"——班级解散了学生还在；组合是"不可分离"——人死了心脏也没了。判断关键：部分能否脱离整体独立存在。能→聚合，不能→组合。

总恍然大悟：五种关系强度递增对应5种"在一起"的程度——临时合作（依赖：今天用一下）→ 长期搭档（关联：常合作）→ 室友（聚合：共享空间但各自独立）→ 夫妻（组合：法律意义的共同体）→ 亲子（继承：血缘关系）。强度越高耦合越紧，改起来越牵连。设计时应优先选弱关系——能用关联不用聚合，能用聚合不用组合，能用组合不用继承。这就是"低耦合"的体现：让对象之间的关系尽可能松散，便于独立修改和测试。

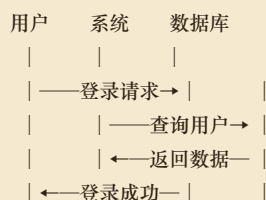
顺序图简介（考点）

顺序图描述对象之间消息交互的时间顺序（从上到下=时间流逝）。

基本元素：

- 对象（顶部矩形）
- 生命线（对象下方的虚线）
- 消息（对象间的箭头，从上到下表示时间顺序）

简单例子（用户登录）：



恍然大悟：顺序图和状态转换图都描述"动态行为"，但角度不同——顺序图看"对象间的交互时序"（谁对谁发什么消息），状态图看"单个对象的状态变化"（自己怎么变）。一个管"交流"，一个管"自转"。

考点聚焦

高频考点清单

1. 功能需求 vs 非功能需求 vs 领域需求的场景判断
2. 三种模型与图的对应（数据→ER、功能→DFD、行为→STD）
3. 用例图包含(include) vs 扩展(extend)
4. 需求验证四维度（一致性、现实性、完整性、有效性）
5. UML图分类（结构图vs行为图）、类图四种关系（依赖/关联/聚合/组合）
6. SA方法的工具（DFD+DD+ER+STD）
7. SRS是需求分析阶段的输出
8. 面向对象基本概念（封装/继承/多态）、聚合vs组合的区分

常见题型

- 场景判断：给定需求描述判断功能/非功能/领域
- 对应关系：三种模型分别用什么图
- 关系辨析：包含vs扩展
- 维度判断：需求验证的四维度

记忆口诀总览

1. 需求获取："访问情原观"（访谈、问卷、情景、原型、观察）
2. 五种获取方法本质："问写演试看"（5种方法对应5种动作）
3. 需求分类："功能做什么，非功做多好，领域看行业"
4. 三种模型："数E功D行状态"（数据→ER，功能→DFD，行为→状态转换图）
5. 需求验证："一现完有"（一致性、现实性、完整性、有效性）
6. 用例图关系："包含必有，扩展可有"
7. SA工具："DFD+DD+ER+STD"（四个工具）
8. 面向对象："类对封继多消息"（类、对象、封装、继承、多态、消息）



9. 三大支柱："封装隔离变化，继承复用代码，多态扩展行为"
 10. UML分类："结构管有什么，行为管怎么动"
 11. 类间关系："依关聚组继"（依赖、关联、聚合、组合、继承，从弱到强）
 12. 类图UML符号："菱形=整体，三角=父类，虚实=强弱"
 13. 聚合vs组合："聚合可分离，组合不可分"（班级-学生 vs 人-心脏）
-

本章小结

本章核心逻辑链：搞清楚做什么（需求分析）→ 从用户获取需求（访谈/问卷/原型等）→ 分类整理（功能/非功能/领域）→ 建立三种模型（数据/功能/行为）→ 输出SRS文档 → 验证需求质量（一致性/现实性/完整性/有效性）。

本章最重要的三个考点：需求分类的场景判断（功能做什么vs非功能做得多好）、三种模型与图的对应（数据ER/功能DFD/行为STD）、用例图包含与扩展的区分（包含必有vs扩展可有）。需求分类是选择题最高频的设坑点，务必分清"做什么"（功能）和"做得多好"（非功能）。本章的DFD和数据字典与第二章衔接，是结构化分析方法的核心工具。

